

Ducks Unlimited University Chapters ETL Pipeline

AIM: The aim of this project is to build an end-to-end automated ETL (Extract, Transform, Load) pipeline that extracts university chapter data for Ducks Unlimited from a public ArcGIS API endpoint, validates and transforms the records into a clean schema, and securely loads them downstream into Google BigQuery.

Key objectives include:

- 1.) Automated Data Integration: Programmatically fetching university chapter records using Python instead of manual data collection.
- 2.) Data Quality & Transformation: Parsing nested JSON structures and spatial attributes to ensure uniform data formats, accurate coordinates, and explicit schema constraints.
- 3.) Cloud Storage: Persisting structured data reliably inside Google BigQuery for reporting and analysis.
- 4.) Software Engineering Best Practices: Incorporating modular script design, automated unit testing with `pytest`, and continuous integration/continuous deployment (CI/CD) workflows via GitHub Actions.

Below are the source file names and target table which I used to migrate the data:

Component Type	Name / Identifier
Source File	src/src_extract.py
Source File	src/src_transform.py
Source File	src/src_bigquery_load.py
Source File	src/src_pipeline.py
Testing File	tests/test_pipeline.py
Testing File	tests/test_transform.py
CI/CD Config	.github/workflows/CI-CD.yml
Cloud Target	Google BigQuery Project (dp-dev-data-pipeline)
Cloud Target	BigQuery Dataset (etl_pipeline)
Cloud Target	BigQuery Table (university_chapters)

The screenshot shows a Windows 10 desktop with a VS Code editor open. The editor displays a GitHub Actions workflow file named `main.yml` for the repository `actions/checkout@v2`. The workflow is configured to run on a self-hosted runner named `dev-ubuntu-pipeline`. The workflow includes a `checkout` step to fetch the code, a `run-python` step to install dependencies and run tests, and a `test` step to run the tests. The workflow is triggered by a `push` event to the `main` branch.

```

name: Run Python Pipeline
on:
  push:
    branches: [ main ]
jobs:
  run-python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Run unit tests
        run: |
          python tests/
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up Python
        uses: actions/setup-python@v2
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Run unit tests
        run: |
          python tests/

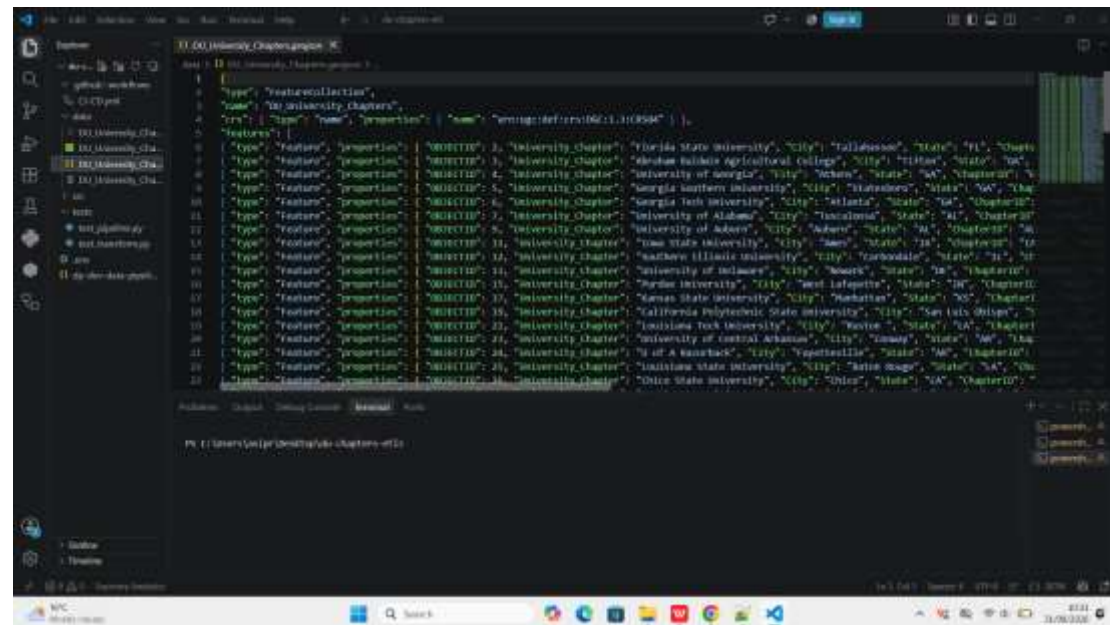
```

The VS Code interface shows the file explorer on the left with the `main.yml` file selected. The bottom status bar indicates the file is open in the `main` branch of the `actions/checkout@v2` repository.

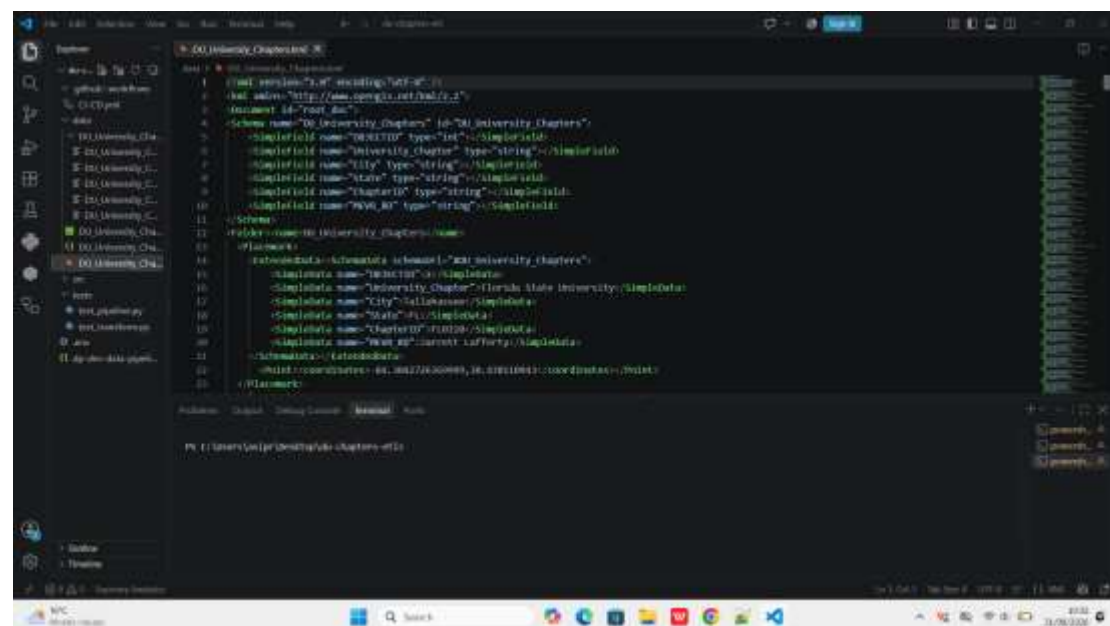
CSV:-

[illegible]

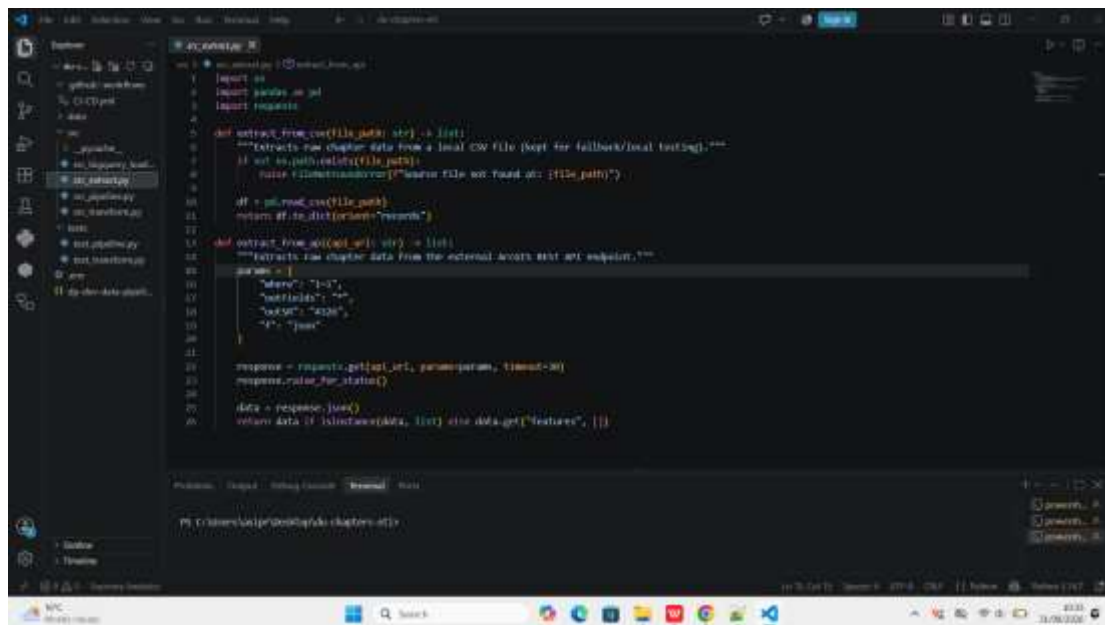
JSON:-



KML:-



Step 3:- Below code reads raw chapter records from a local file as a fallback or testing option. Connects to the external ArcGIS REST API using `requests` with parameters to fetch raw JSON feature data.

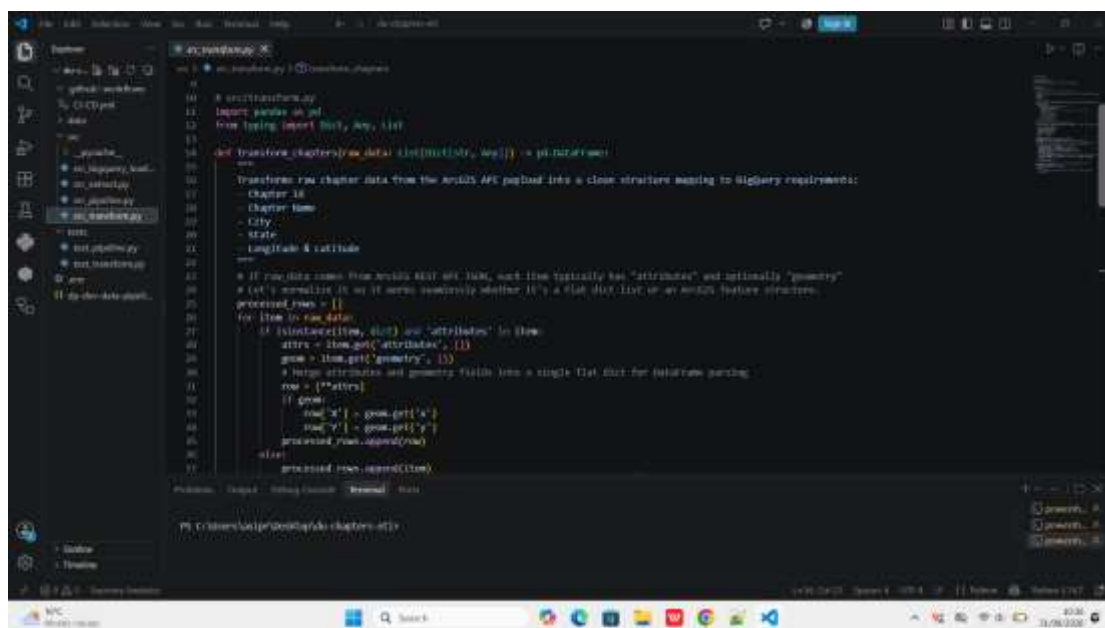


```

1 import os
2 import pandas as pd
3 import requests
4
5 def extract_from_csv(file_path: str) -> list:
6     """Extracts raw chapter data from a local CSV file (kept for fallback/local testing)."""
7     if not os.path.exists(file_path):
8         raise FileNotFoundError(f"Source file not found at: {file_path}")
9
10    df = pd.read_csv(file_path)
11    return df.to_dict(orient="records")
12
13 def extract_from_api(api_url: str) -> list:
14     """Extracts raw chapter data from the external ArcGIS REST API endpoint."""
15     params = {
16         "where": "1=1",
17         "outfields": "*",
18         "outSR": "4326",
19         "f": "json"
20     }
21
22     response = requests.get(api_url, params=params, timeout=30)
23     response.raise_for_status()
24
25     data = response.json()
26     return data if isinstance(data, list) else data.get("features", [])
27
28 # Example usage:
29 # raw_data = extract_from_csv('chapters.csv')
30 # raw_data = extract_from_api('https://api.arcgis.com/v1/featureset/chapters')

```

Step 4:- Normalizes raw API payloads by separating or parsing nested `attributes` and spatial `geometry` fields. Extracts longitude (`x`) and latitude (`y`) coordinates to map structural requirements for downstream storage. Outputs a clean Pandas DataFrame mapping core fields like Chapter ID, Name, City, and State.



```

1 from typing import Dict, Any, List
2
3 def transform_chapters(raw_data: List[Dict[str, Any]]) -> pd.DataFrame:
4     """
5     Transform raw chapter data from the ArcGIS API payload into a clean structure mapping to BigQuery requirements:
6     - Chapter ID
7     - Chapter Name
8     - City
9     - State
10    - Longitude & Latitude
11    """
12    # If raw data comes from ArcGIS REST API, each item typically has "attributes" and optionally "geometry"
13    # Let's normalize it so it works consistently whether it's a flat dict/list or an ArcGIS feature structure.
14    processed_rows = []
15
16    for item in raw_data:
17        if isinstance(item, dict) and "attributes" in item:
18            attrs = item.get("attributes", {})
19            geom = item.get("geometry", {})
20            # Merge attributes and geometry fields into a single flat dict for dataframe parsing
21            row = {"attrs": attrs}
22            if geom:
23                row["x"] = geom.get("x")
24                row["y"] = geom.get("y")
25            processed_rows.append(row)
26        else:
27            processed_rows.append(item)
28
29    return pd.DataFrame(processed_rows)
30
31 # Example usage:
32 # normalized_data = transform_chapters(raw_data)
33 # normalized_data.to_csv('normalized_chapters.csv')

```

Step 5:- In the below code:

- Client Connection: Initializes a Google Cloud BigQuery client to connect to your project.
- Schema Enforcement: Defines strict data types and constraints (`STRING`, `FLOAT`, `REQUIRED`, `NULLABLE`) for all chapter attributes.
- Table Overwrite: Loads the Pandas DataFrame into BigQuery using `WRITE\_TRUNCATE` to refresh the target table.

```

import os
from google.cloud import bigquery
from google.cloud.exceptions import BadRequest
import pandas as pd

def load_to_bigquery(df, dataset_id, table_id, schema):
    """
    Loads the transformed pandas dataframe into Google BigQuery.
    """
    client = bigquery.Client()
    full_table_id = f"{client.project}.{dataset_id}.{table_id}"

    # Define explicit schema to ensure data integrity
    job_config = bigquery.LoadJobConfig(
        write_disposition=bigquery.WriteDisposition.WRITE_TRUNCATE,
        schema=schema
    )

    bigquery.SchemaField("chapter_id", "STRING", mode="REQUIRED"),
    bigquery.SchemaField("chapter_name", "STRING", mode="REQUIRED"),
    bigquery.SchemaField("city", "STRING", mode="NULLABLE"),
    bigquery.SchemaField("state", "STRING", mode="NULLABLE"),
    bigquery.SchemaField("longitude", "FLOAT", mode="REQUIRED"),
    bigquery.SchemaField("latitude", "FLOAT", mode="REQUIRED"),

    try:
        load_job = client.load_table_from_dataframe(df, full_table_id, job_config=job_config)
        load_job.result()  # Wait for the job to complete
    except BadRequest as e:
        print(f"Error loading data: {e}")

# Example usage
df = pd.DataFrame(...)
load_to_bigquery(df, "my_dataset", "chapters", schema)

```

Step 6:- In below pipeline file:-

- Environment Configuration: Loads environment variables using `load\_dot\_env()` and sets configurations like data source type (`api` or `csv`), dataset ID, and table ID.
- Orchestration Execution: Acts as the master control script that sequentially coordinates the workflow by calling the extraction, transformation, and loading functions.

```

import os
from dotenv import load_dotenv
from src.extract import extract_from_api, extract_from_csv
from src.transform import transform_chapters
from src.load import load_to_bigquery

# Load environment variables from a .env file if present
load_dotenv()

def run_pipeline():
    # Toggle between 'api' or 'csv' via environment variable (defaults to 'api' to meet requirements)
    source_type = os.getenv("SOURCE_TYPE", "api").lower()

    # Dataset ID and Table ID
    dataset_id = os.getenv("BIGQUERY_DATASET_ID", "my_dataset")
    table_id = os.getenv("BIGQUERY_TABLE_ID", "chapters")

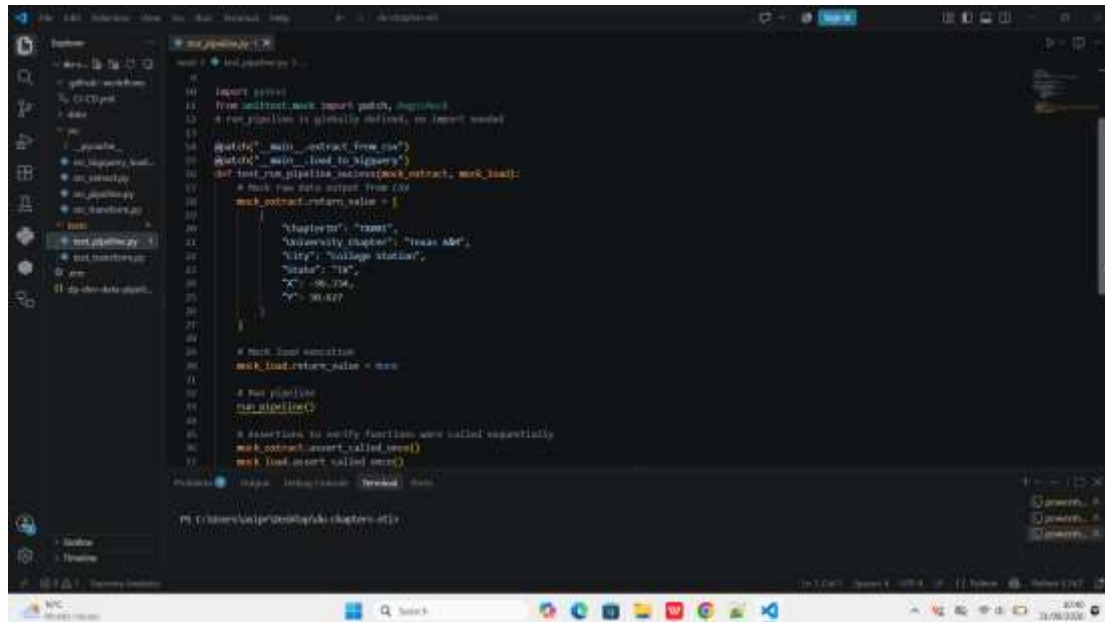
    print(f"[INFO] Step 1: Extracting data...")
    if source_type == "csv":
        data_path = os.getenv("DATA_SOURCE_PATH", "data/university_chapters.csv")
        print(f"[INFO] Extracting from local src: {data_path}")
        raw_data = extract_from_csv(data_path)
    else:
        api_url = os.getenv(
            "API_URL",
            "https://services.google.com/fh/files/doc/txt/university_chapters.txt?authuser=0&as=csv"
        )
        print(f"[INFO] Extracting from remote API: {api_url}")
        raw_data = extract_from_api(api_url)

    # Transform and Load
    transformed_data = transform_chapters(raw_data)
    load_to_bigquery(transformed_data, dataset_id, table_id)

```

Step 7:- In testing

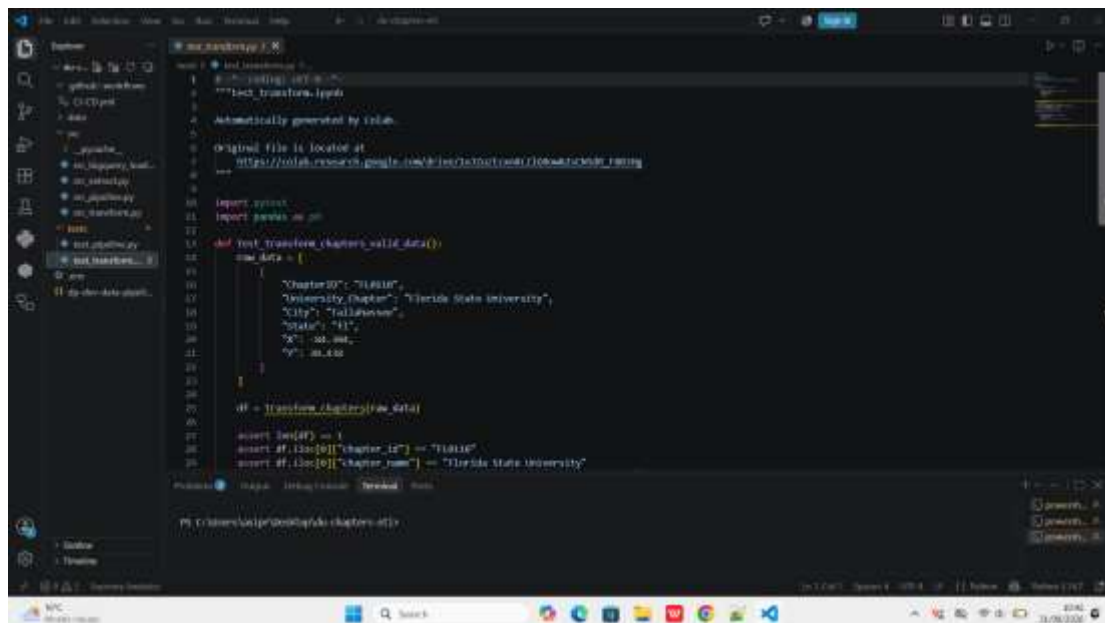
- Pipeline Mocking: Uses `unittest.mock` (`patch`) to isolate the orchestration test by mocking external dependencies like extraction and loading functions.
- Execution & Assertion: Simulates a successful run by passing mock chapter data, executing `run\_pipeline()`, and asserting that both extract and load steps were called sequentially.



```
10 import pytest
11 from unittest.mock import patch, MagicMock
12 # run_pipeline is globally defined, no import needed
13
14 @patch('main._extract', return_value=mock_data)
15 @patch('main._load_to_splaweb')
16 def test_run_pipeline_sequential(mock_extract, mock_load):
17     # Mock raw data output from /api
18     mock_extract.return_value = [
19         {
20             "chapter_id": "118110",
21             "university_chapter": "Florida State University",
22             "city": "Tallahassee",
23             "state": "FL",
24             "lat": 30.438,
25             "lon": -84.281
26         }
27     ]
28
29     # Mock load operation
30     mock_load.return_value = None
31
32     # Run pipeline
33     run_pipeline()
34
35     # Assertions to verify functions were called sequentially
36     mock_extract.assert_called_once()
37     mock_load.assert_called_once()
```

Step 8:-

- Transformation Testing**: Tests the `transform\_chapters` function using sample raw chapter data to verify proper parsing.
- Assertion Checks**: Validates that the resulting DataFrame has the correct row count (`len(df) == 1`) and accurately maps specific fields like chapter ID and chapter name.



```
1 # Test transform_chapters
2 """test transform_chapters.py"""
3
4 Automatically generated by robot.
5
6 Original file is located at
7 https://robotframework.google.cloud/dev/testdata/robotframework-factory
8
9 import pytest
10 import pandas as pd
11
12 def test_transform_chapters_valid_data():
13     raw_data = [
14         {
15             "chapter_id": "118110",
16             "university_chapter": "Florida State University",
17             "city": "Tallahassee",
18             "state": "FL",
19             "lat": 30.438,
20             "lon": -84.281
21         }
22     ]
23
24     df = transform_chapters(raw_data)
25
26     assert len(df) == 1
27     assert df.iloc[0]["chapter_id"] == "118110"
28     assert df.iloc[0]["chapter_name"] == "Florida State University"
```


Step 9:- When I run command in terminal:-

Python -m src.src_pipeline it will load 136 records in my BigQuery table(university_chapters) after processing

The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled 'data_pipeline.py' and contains a function 'run_pipeline()' that performs the following steps:

- Extracting data from a CSV file 'chapters.csv' using 'pd.read_csv()'.
- Transforming and validating data by cleaning the 'df' object using 'clean_df = transform_chapters(raw_data)'.
- Loading data into a BigQuery table 'chapters' using 'load_to_bigquery()' with parameters 'raw_data', 'big_query_table', 'big_query_dataset', and 'big_query_project'.
- Final message: 'ETL Pipeline executed successfully.'

The output shows the execution of these steps, including the file path 'C:\Users\user\AppData\Local\Python\Python38-32\Scripts\python.exe' and the command 'python -m data_pipeline.py'.

Step 10:- Total 136 records were loaded in BigQuery table

The screenshot shows the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo and a search bar. Below the navigation bar, there's a sidebar with a menu containing options like Home, Starred, Shared with me, Job history, and a list of datasets and connections. The main area displays the 'university_chapters' table in a preview view. The table has 7 columns: row, chapter_id, chapter_name, city, state, longitude, and latitude. It contains 15 rows of data, listing university chapters from various locations such as Auburn, Jacksonville, Tuskegee, Fayetteville, Jacksonville, Conway, Arkansas Tech, Marietta, Magnolia, Tampa, Ohio, San Luis Obispo, Fresno, and Colorado Springs.